



Recibimos una lista de edificios ordenados de izquierda a derecha con dos datos para cada uno, por un lado, el ancho W_i de cada edificio y por el otro el alto H_i . Notar que los edificios están ordenados de izquierda a derecha.

Buscamos la longitud de la **máxima subsecuencia ascendente**, es decir, la subsecuencia creciente de edificios que ocupe el mayor espacio horizontal.

- a) Diseñar un algoritmo de PD que resuelva el problema en $O(N^2)$ tiempo y espacio.
- b) Optimizar el algoritmo anterior para que funcione con $O(N)$ memoria auxiliar.
- c) (Opcional) Utilizar una estructura de datos como **Segment Tree** o **Feenwick Tree** para conseguir una complejidad temporal $O(N \log N)$.

Guirnaldas

26. Este problema fue extraído de **Codeforces (1286A)**

Vladimir ama decorar su árbol de navidad. Tiene n guirnaldas, numeradas de 1 a n . Para decorar el árbol Vladimir debe colocar las guirnaldas en una secuencia. La **complejidad** de una secuencia s de guirnaldas se define como el número de pares de guirnaldas adyacentes con distinta paridad. Vladimir tiene una secuencia parcialmente decorada de guirnaldas, s . Los lugares libres se indican con un valor 0. Se pide completar la secuencia s con las guirnaldas restantes, de modo que la complejidad de la secuencia resultante sea mínima.

- a) Diseñar un algoritmo de PD que resuelva el problema en $O(N^3)$ tiempo y espacio.
- b) Optimizar el algoritmo anterior para que funcione en $O(N^2)$ tiempo y espacio.
- c) Optimizar el algoritmo anterior para que funcione en $O(N^2)$ tiempo y $O(N)$ espacio.

Golosos ($\equiv$ avariciosos $\equiv$ *greedy*)

SumaSelectiva

27. Dado un conjunto X con $|X| = n$ y un entero $k \leq n$ queremos encontrar el máximo valor que pueden sumar los elementos de un subconjunto S de X de tamaño k . Más formalmente, queremos calcular $\max_{S \subseteq X, |S|=k} \sum_{s \in S} s$.

- a) Proponer un algoritmo *greedy* que resuelva el problema, demostrando su correctitud. Extender el algoritmo para que también devuelva uno de los subconjuntos S que maximiza la suma.
- b) Dar una implementación del algoritmo del inciso **a)** con complejidad temporal $O(n \log n)$.
- c) Dar una implementación del algoritmo del inciso **a)** con complejidad temporal $O(n \log k)$.

SumaGolosa

28. Queremos encontrar la suma de los elementos de un multiconjunto de números naturales. Cada suma se realiza exactamente entre dos números x e y y tiene costo $x + y$.

Por ejemplo, si queremos encontrar la suma de $\{1, 2, 5\}$ tenemos 3 opciones:



- $1 + 2$ (con costo 3) y luego $3 + 5$ (con costo 8), resultando en un costo total de 11;
- $1 + 5$ (con costo 6) y luego $6 + 2$ (con costo 8), resultando en un costo total de 14;
- $2 + 5$ (con costo 7) y luego $7 + 1$ (con costo 8), resultando en un costo total de 15.

Queremos encontrar la forma de sumar que tenga costo mínimo, por lo que en nuestro ejemplo la mejor forma sería la primera.

- Explicitar una estrategia golosa para resolver el problema.
- Demostrar que la estrategia propuesta resuelve el problema.
- Implementar esta estrategia en un algoritmo iterativo. **Nota:** el mejor algoritmo simple que conocemos tiene complejidad $\mathcal{O}(n \log n)$ y utiliza una estructura de datos que implementa una secuencia ordenada.

Deadlines

29. Se tiene un conjunto de n tareas $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$, donde cada tarea t_i tiene asociado un *deadline* $d_i \in \mathbb{Z}^+$ que representa el tiempo límite antes del cual debe completarse la tarea. Debemos respetar las siguientes restricciones:

- Cada tarea requiere exactamente **una unidad de tiempo** para completarse.
- Solo se puede ejecutar **una tarea a la vez** (no existe paralelismo).
- El tiempo comienza en $t = 0$ y avanza en unidades discretas: $t \in \{0, 1, 2, \dots\}$.
- Una tarea t_i se considera completada exitosamente si y solo si termina su ejecución en un tiempo $f_i \leq d_i$, donde f_i es el tiempo de finalización de la tarea.
- Una vez iniciada una tarea, debe completarse sin interrupciones (no hay *preemption*).
- Las tareas pueden ejecutarse en cualquier orden.

Por ejemplo, considere $n = 4$ tareas con *deadlines* $D = \{2, 1, 3, 2\}$. Una solución óptima sería: $O = \{t_2, t_1, t_3\}$. Esta planificación completa 3 tareas exitosamente. La tarea t_4 no puede ejecutarse sin violar su *deadline*.

- Proponer un algoritmo *greedy* que determine una planificación (orden de ejecución) que **maximice el número total de tareas** que pueden completarse respetando sus respectivos *deadlines*.
- Demuestre la optimalidad.

RutaEficiente

30. Tomás quiere viajar de Buenos Aires a Mar del Plata en su flamante Renault 12. Como está preocupado por la autonomía de su vehículo, se tomó el tiempo de anotar las distintas estaciones de servicio que se encuentran en el camino. Modeló el mismo como un segmento de 0 a M , donde Buenos Aires está en el kilómetro 0, Mar del Plata en el M , y las distintas estaciones de servicio están ubicadas en los kilómetros $0 = x_1 \leq x_2 \leq \dots x_n \leq M$.



Razonablemente, Tomás quiere minimizar la cantidad de paradas para cargar nafta. Él sabe que su auto es capaz de hacer hasta C kilómetros con el tanque lleno, y que al comenzar el viaje este está vacío.

- a) Proponer un algoritmo *greedy* que indique cuál es la cantidad mínima de paradas para cargar nafta que debe hacer Tomás, y que aparte devuelva el conjunto de estaciones en las que hay que detenerse. Probar su correctitud.
- b) Dar una implementación de complejidad temporal $O(n)$ del algoritmo del inciso a).

Producto Escalar

31. Dados dos vectores $v = (v_1, v_2, \dots, v_n)$ y $w = (w_1, w_2, \dots, w_n)$ en $\mathbb{R}^n$, queremos encontrar permutaciones $\sigma, \tau : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$ de sus coordenadas que minimicen el producto escalar entre los vectores reordenados. Es decir:

$$\langle v_\sigma, w_\tau \rangle = \sum_{i=1}^n v_{\sigma(i)} \cdot w_{\tau(i)}$$

sea mínimo, donde $v_\sigma = (v_{\sigma(1)}, \dots, v_{\sigma(n)})$ y $w_\tau = (w_{\tau(1)}, \dots, w_{\tau(n)})$

- a) Proponer un algoritmo *greedy* que encuentre las permutaciones óptimas.
- b) Demostrar la optimalidad.

División Pandémica

32. En medio de una pandemia, la Escuela de Aulas Grandes y Ventiladas quiere implementar un protocolo especial de distanciamiento social que tenga en cuenta que la escuela no tienen restricciones de espacio. El objetivo es separar a cada curso en dos subcursos a fin de reducir la cantidad de pares de estudiantes que sean muy cercanos, dado que se estima que estos estudiantes tienen dificultades para respetar tan buscado distanciamiento. Para este fin, en el protocolo se estableció que cada curso que tenga c parejas de estudiantes cercanos tiene que dividirse en dos subcursos, cada uno de los cuales puede tener a lo sumo $c/2$ parejas de estudiantes cercanos. Notar que no importa si un subcurso queda con más estudiantes que otro.

Formalmente, para cada curso contamos con un conjunto de estudiantes E y su conjunto C de pares de estudiantes cercanos. Luego, una partición (A, B) de E es una *solución factible para* (E, C) cuando $|(A \times A) \cap C| \leq |C|/2$ y $|(B \times B) \cap C| \leq |C|/2$. Por ejemplo, si $E = \{1, 2, 3, 4\}$ y $C = \{1-2, 2-3, 3-4\}$, entonces $(\{1, 3, 4\}, \{2\})$ y $(\{2, 4\}, \{1, 3\})$ son soluciones factibles.

- a) Especificar el problema descrito definiendo cuál es la instancia (i.e. cuáles son los datos de entrada y qué condiciones satisfacen) y cuál es el resultado esperado (i.e., cuáles son los datos de salida y qué condiciones satisfacen).
- b) Demostrar que para toda instancia existe un resultado esperado que satisface las condiciones definidas por el protocolo. **Ayuda:** hacer inducción en la cantidad de estudiantes. Para el paso inductivo, considerar que si les estudiantes se asignan iterativamente a los subcursos, entonces conviene enviar a cada estudiante al subcurso que tenga la menor cantidad de estudiantes cercanos a él.



- c) A partir de la demostración del inciso anterior, diseñar un algoritmo que encuentre una solución factible en tiempo lineal en función del tamaño de la entrada definido en el inciso a).

MaxMex

33. Se define la función $mex : \mathcal{P}(\mathbb{N}) \rightarrow \mathbb{N}$ como

$$mex(X) = \min\{j : j \in \mathbb{N} \wedge j \notin X\}$$

Intuitivamente, mex devuelve, dado un conjunto X , el menor número natural que no está en x . Por ejemplo, $mex(\{0, 1, 2\}) = 3$, $mex(\{0, 1, 3\}) = 2$ y $mex(\{1, 2, 3, \dots\}) = 0$.

Dado un vector de número $a_1 \dots a_n$ queremos encontrar la permutación $b_1 \dots b_n$ de los mismos que maximize

$$\sum_{i=1}^n mex(\{b_1 \dots b_i\})$$

Por ejemplo, si el vector es $\{3, 0, 1\}$ podemos ver que la mejor permutación es $\{0, 1, 3\}$, que alcanza un valor de

$$mex(\{0\}) + mex(\{0, 1\}) + mex(\{0, 1, 3\}) = 1 + 2 + 2 = 5$$

- a) Proponer un algoritmo *greedy* que resuelva el problema y demostrar su correctitud. **Ayuda:** ¿Cuál el máximo valor que puede tomar $mex(X)$ si X tiene n elementos? Si $X \subseteq Y$, ¿Qué pasa con los valores $mex(X)$ y $mex(Y)$?
- b) Dar una implementación del algoritmo del inciso anterior con complejidad temporal $O(n)$.

CacheOpt

34. Como los accesos a memoria RAM son lentos en comparación al trabajo propio del CPU, es común que se coloquen memorias intermedias más diminutas y de alta velocidad entre ambas unidades, las cuales llamamos *cachés*. Cuando un programa se ejecuta y hace una consulta a la memoria por cierta posición r primero se verifica si la posición r está cargada en la caché, en cuyo caso el CPU la puede obtener sin tener que hacer el acceso a la RAM. Cuando esto ocurre, decimos que ocurre un *caché hit*. En cambio, si la posición r no está en la caché, esta se busca a memoria, se carga en la caché, y luego se la informa al CPU. A este evento se lo conoce como *caché miss*.

Como la caché es más chica que la memoria RAM es inevitable que eventualmente ocurra un *caché miss* y que la caché este llena. En ese caso, la caché debe decidir qué información va a desechar para darle lugar a la nueva entrada. Naturalmente, se busca minimizar la cantidad de *misses* de los siguientes accesos.

El problema de *Off-line* caching consiste en determinar, dada una caché C de tamaño k y una lista de n requests $R = \{r_1, r_2, \dots, r_n\}^\dagger$ a posiciones de memoria, qué decisión debe tomar en

[†]Sin pérdida de generalidad respecto al problema, podemos asumir que $1 \leq r_i \leq n$ para todo i .



cada paso la caché para minimizar la cantidad de *misses*. Por ejemplo, si $k = 2$ y $R = \{1, 2, 3, 1\}$ entonces:

- La primera consulta es un *miss*, pero como hay lugar en la caché (empieza vacía) se carga la posición 1 a C ($C = \{1\}$).
- Con la segunda consulta pasa lo mismo, por lo que la caché queda en el estado $C = \{1, 2\}$.
- En la tercera consulta la caché esta llena, por lo que se debe desechar alguna entrada. Notemos que si se desecha 1 entonces la cuarta consulta dará otro *miss*, mientras que si se desecha 2 entonces habrá un *hit*.

Una política posible para decidir qué elemento desechar es la *furthest-in-future*: se desecha aquella posición r cuyo siguiente acceso es el más lejano (o bien, que no tiene un siguiente acceso).

- a) **Opcional:** Definir una función recursiva $f(i, mem)$ que tome un índice y un estado de la memoria y devuelva la mínima cantidad de *cache misses* que deben ocurrir para procesar todas las consultas $\{r_i, r_{i+1}, \dots, r_n\}$ si el estado actual de la memoria es mem . ¿Con qué llamado se resuelve el problema? Estudiar la superposición de subproblemas y explicar en qué casos vale la pena memorizar.
- b) Probar que la política *furthest-in-future* es óptima (es decir, que minimiza la cantidad de *misses*). **Ayuda:** Dada una serie de decisiones, probar que si en un paso no se sigue la política *furthest* entonces podemos alterar ese paso para que sí la siga sin afectar la cantidad de *misses*.
- c) Dar un algoritmo con complejidad temporal $O(n \log(k))$ que informe qué decisión debe tomar la caché en cada paso para minimizar la cantidad de *misses*.

Parejas de Baile (problema difícil)

35. Tenemos dos conjuntos de personas y para cada persona sabemos su habilidad de baile. Queremos armar la máxima cantidad de parejas de baile, sabiendo que para cada pareja debemos elegir exactamente una persona de cada conjunto de modo que la diferencia de habilidad sea menor o igual a 1 (en módulo). Además, cada persona puede pertenecer a lo sumo a una pareja de baile. Por ejemplo, si tenemos un multiconjunto con habilidades $\{1, 2, 4, 6\}$ y otro con $\{1, 5, 5, 7, 9\}$, la máxima cantidad de parejas es 3. Si los multiconjuntos de habilidades son $\{1, 1, 1, 1, 1\}$ y $\{1, 2, 3\}$, la máxima cantidad es 2.

- a) Considerando que ambos multiconjuntos de habilidades están ordenados en forma creciente, observar que la solución se puede obtener recorriendo los multiconjuntos en orden para realizar los emparejamientos.
- b) Diseñar un algoritmo goloso basado en a) que recorra una única vez cada multiconjunto. Explicitar la complejidad temporal y espacial auxiliar.
- c) Demostrar que el algoritmo dado en b) es correcto.



Ejercicios integradores

Invitación Estratégica

36. El problema de la fiesta consiste en determinar un conjunto de invitados que no tengan conflictos entre sí y que sea de cardinalidad máxima. Formalmente, dado un conjunto V de posibles invitados y un conjunto E de conflictos, formados por pares no ordenados de V , queremos encontrar un subconjunto $S \subseteq V$ de cardinalidad máxima entre aquellos que cumplen que $\{v, w\} \notin E$ para todo par $v, w \in S$. Por ejemplo, si $S = \{1, 2, 3, 4, 5\}$ y $E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{4, 5\}\}$, entonces una solución es $S = \{1, 3, 5\}$, ya que no se puede invitar a ningún conjunto de 4 personas. Vamos a suponer que los posibles invitados se representan con el conjunto $V = \{1, \dots, n\}$ para algún $n \geq 0$ (el caso $n = 0$ es válido y representa el conjunto $V = \emptyset$).

- a) Decimos que $S \subseteq \mathbb{N}$ y $W \subseteq \mathbb{N}$ son *compatibles* cuando $S \subseteq V$ es un conjunto posible de invitados y ningún elemento de $W \subseteq V \setminus S$ tiene un conflicto con algún elemento de S . En el ejemplo anterior, $S = \{1\}$ y $W = \{4, 5\}$ son compatibles pero $S = \{1, 4\}$ y $W = \{2\}$ no lo son. Sea $\mathcal{V}$ el conjunto de subconjuntos de V . Escribir una función recursiva $\text{fs}: \mathcal{V} \times \mathcal{V} \rightarrow \mathcal{V}$ tal que, dados S y W compatibles, $\text{fs}(S, W)$ retorne un conjunto de invitados de máxima cardinalidad que contenga a S . (Notar que la llamada recursiva debe garantizar la compatibilidad). **Ayuda:** considerar dos posibilidades: no invitar a $w \in W$, o invitar a $w \in W$ y no invitar a nadie que tenga un conflicto con w .
- b) En base a [a\)](#), implementar un algoritmo recursivo de *backtracking* para resolver el problema de la fiesta basado en las siguientes ideas:
 - cada solución parcial es un conjunto $S \subseteq V$ que no contiene invitados con conflictos.
 - a cada nodo del árbol de *backtracking* se le asocia un conjunto $W \subseteq V$ compatible con S de posibles invitados.
 - para la extensión, se consideran dos posibilidades: o bien no se invita a $w \in W$ o bien se invita a w y se eliminan de W todos los otros elementos que estén en conflicto con w .
- c) Escribir los tres primeros niveles del árbol de *backtracking* resultante de la implementación anterior.
- d) Describir una regla de optimalidad para poder podar el árbol e incluirla en la implementación de [b\)](#).
- e) ¿Se le ocurre una forma de escribir una función recursiva $\text{fs}(V, S, i)$ que, en analogía con el inciso [a\)](#) del Ejercicio 11, determine el conjunto de invitados óptimo que incluya a $S \subseteq \{1, \dots, i-1\}$ y que se obtenga agregando sólo invitados de $\{i, \dots, n\}$? ¿Cuál es el problema? ¿Se le ocurre alguna manera de escapar a este problema?
- f) Considerando la función fs (definida en [a\)](#)) y el inciso anterior, observar que la cantidad posible de instancias es $\Omega(2^n)$. Concluir que la función fs no tiene la propiedad de superposición de subproblemas para el caso general del problema de la fiesta.

Selección de Actividades

37. Dado un conjunto de actividades $\mathcal{A} = \{A_1, \dots, A_n\}$, el problema de selección de actividades consiste en encontrar un subconjunto de actividades $\mathcal{S}$ de cardinalidad máxima, tal que ningún



par de actividades de $\mathcal{S}$ se solapan en el tiempo. Cada actividad A_i se realiza en algún intervalo de tiempo (s_i, t_i) , siendo $s_i \in \mathbb{N}$ su momento inicial y $t_i \in \mathbb{N}$ su momento final. Suponemos que $1 \leq s_i < t_i \leq 2n$ para todo $1 \leq i \leq n$.

- a) Considerar la siguiente analogía con el problema de la fiesta: cada posible actividad es un invitado y dos actividades pueden “invitarse” a la fiesta cuando no se solapan en el tiempo. A partir de esta analogía, proponga un algoritmo de *backtracking* para resolver el problema de selección de actividades. ¿Cuál es la complejidad del algoritmo?
- b) Supongamos que $\mathcal{A}$ está ordenado por orden de comienzo de la actividad, i.e., $s_i \leq s_{i+1}$ para todo $1 \leq i < n$. Escribir una función recursiva $\text{act}(\mathcal{A}, \mathcal{S}, i)$ que encuentre el conjunto máximo de actividades seleccionables que contenga a $\mathcal{S} \subseteq \{A_1, \dots, A_{i-1}\}$ y que se obtenga agregando únicamente actividades de $\{A_i, \dots, A_n\}$. **Para reflexionar:** ¿por qué se puede definir act en este caso y no en el inciso e) del Ejercicio 36?
- c) Implementar un algoritmo de programación dinámica para el problema de selección de actividades que se base en la función del inciso b). ¿Cuál es su complejidad temporal y cuál es el espacio extra requerido?
- d) Considerar la siguiente estrategia golosa para resolver el problema de selección de actividades: elegir la actividad cuyo momento final sea lo más temprano posible, de entre todas las actividades que no se solapan con las actividades ya elegidas. Demostrar que un algoritmo goloso que implementa la estrategia anterior es correcto. **Ayuda:** demostrar por inducción que la solución parcial $B_1, \dots, B_i$ que brinda el algoritmo goloso en el paso i se puede extender a una solución óptima. Para ello, suponga en el paso inductivo que $B_1, \dots, B_i, B_{i+1}$ es la solución golosa y que $B_1, \dots, B_i, C_{i+1}, \dots, C_j$ es la extensión óptima que existe por inducción y muestre que $B_1, \dots, B_{i+1}, C_{i+2}, \dots, C_j$ es una extensión óptima de $B_1, \dots, B_{i+1}$.
- e) Mostrar una implementación del algoritmo cuya complejidad temporal sea $\mathcal{O}(n)$.